Informatique Théorique

CM1: Diviser pour reigner

Florian Bridoux

Polytech Nice Sophia

2025-2026

Table des matières

1. Introduction
2. Élément majoritaire
3. Tri fusion
4. Master theorem

Table des matières

1. Introduction
2. Élément majoritaire
3. Tri fusion
4. Master theorem

Diviser pour régner

Étapes :

1. Diviser un problème en sous-problèmes qui sont des instances plus petites du même problème
2. Résoudre récursivement ces sous-problèmes
3. Combiner ces réponses pour obtenir la solution au problème

```
fonction f(Instance I)
    si taille(I) petite :
        resoudre directement
    I1, I2 = diviser(I)
    R1 = f(I_1)
    R2 = f(I_2)
    return combiner(R1, R2)
```

Avantages

- Simplification conceptuelle
- Accélération
  - réduire rapidement (de manière exponentielle) l'espace des problèmes
  - exploiter les points communs dans les solutions des sous-problèmes
- Parallélisation

Table des matières

1. Introduction
2. Élément majoritaire
3. Tri fusion
4. Master theorem

Problème de l'élément majoritaire

Définition :
Dans une liste de taille n, un élément est *majoritaire* si il apparaît strictement plus de $n/2$ fois.

Exemple :
$[a, b, a, a, c, a] \rightarrow a$ est majoritaire car elle apparaît 4 fois sur 6.

Idée de résolution : Diviser pour régner

- Diviser la liste en deux moitiés
- Résoudre récursivement pour chaque moitié
- Combiner les résultats :
  - Si les deux moitiés ont le même candidat majoritaire, c'est lui
  - Sinon, compter les occurrences dans la liste entière pour déterminer le vrai majoritaire

Idée de résolution : Diviser pour régner

Implémentation :

```
def majorite(lst):
    if len(lst) == 1:
        return lst[0]
    gauche = majorite(lst[:len(lst)//2])
    droite = majorite(lst[len(lst)//2:])
    if gauche == droite:
        return gauche
    else:
        if lst.count(gauche) > len(lst)//2:
            return gauche
        if lst.count(droite) > len(lst)//2:
            return droite
        return None
```

Table des matières

1. Introduction
2. Élément majoritaire
3. Tri fusion
4. Master theorem

Tri fusion

L'algorithme peut être décrit récursivement

- On découpe en deux parties à peu près égales les données à trier
- On trie les données de chaque partie
- On fusionne les deux parties

La récursivité s'arrête car on finit par arriver à des listes composées d'un seul élément et le tri est alors trivial

Fusion de deux tableaux triés

On peut fusionner deux listes triées en une seule liste triée en temps linéaire !







Fusion de deux tableaux triés

On peut fusionner deux listes triées en une seule liste triée en temps linéaire !







Fusion de deux tableaux triés

On peut fusionner deux listes triées en une seule liste triée en temps linéaire !

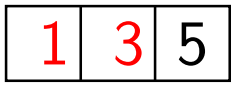




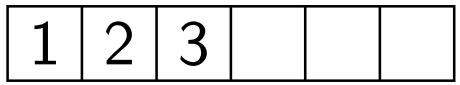


Fusion de deux tableaux triés

On peut fusionner deux listes triées en une seule liste triée en temps linéaire !

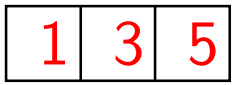




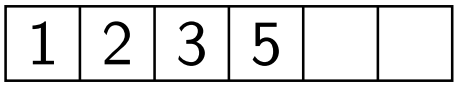


Fusion de deux tableaux triés

On peut fusionner deux listes triées en une seule liste triée en temps linéaire !

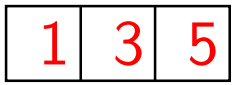




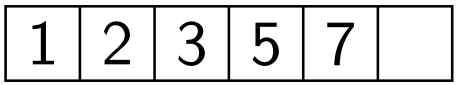


Fusion de deux tableaux triés

On peut fusionner deux listes triées en une seule liste triée en temps linéaire !

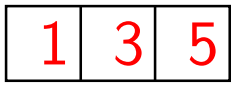




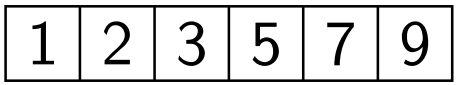


Fusion de deux tableaux triés

On peut fusionner deux listes triées en une seule liste triée en temps linéaire !



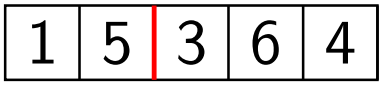




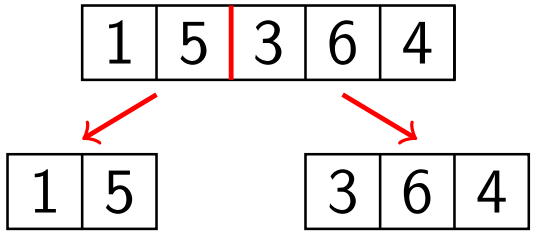
Tri fusion

1	5	3	6	4

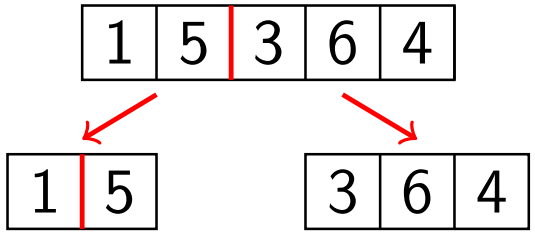
Tri fusion



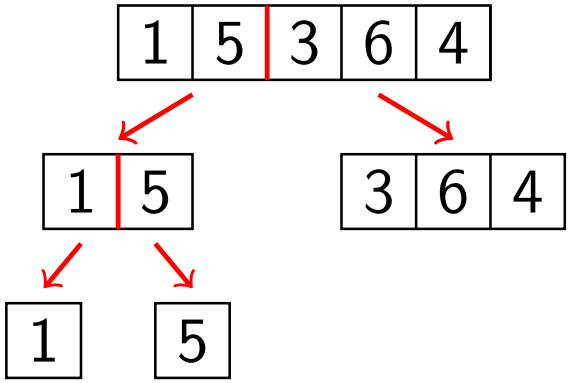
Tri fusion



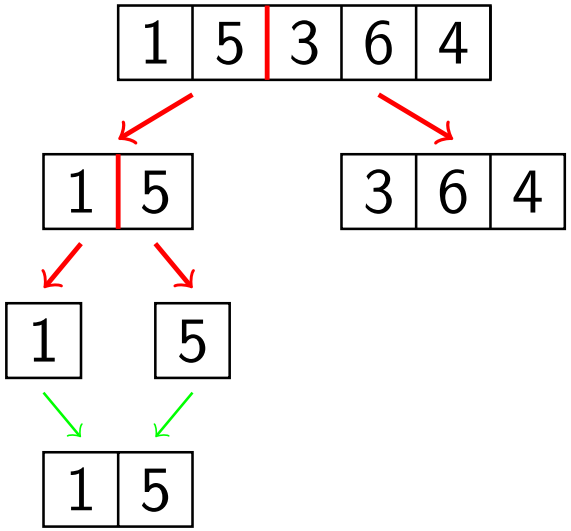
Tri fusion



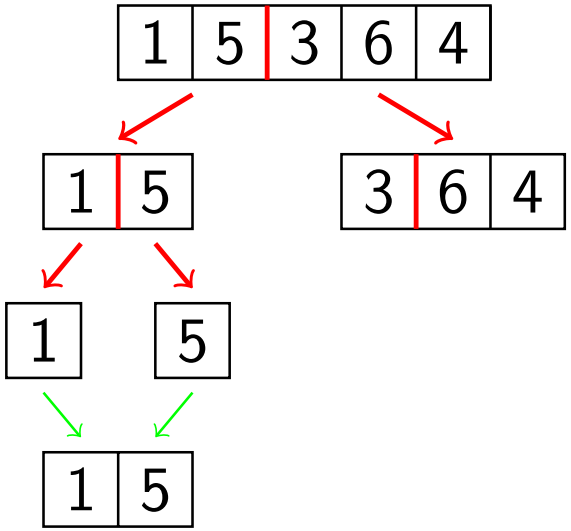
Tri fusion



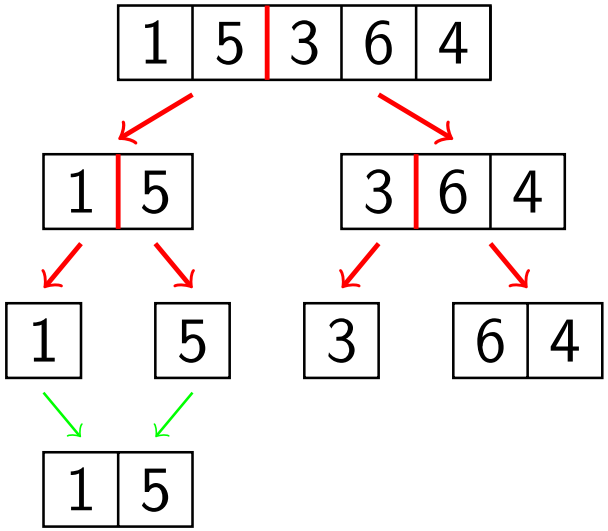
Tri fusion



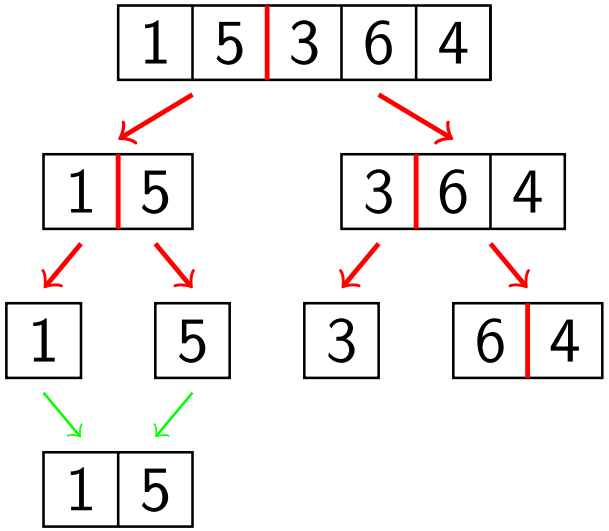
Tri fusion



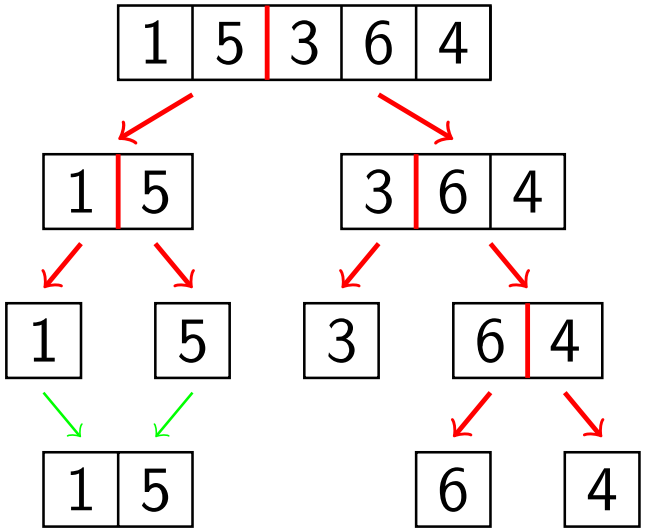
Tri fusion



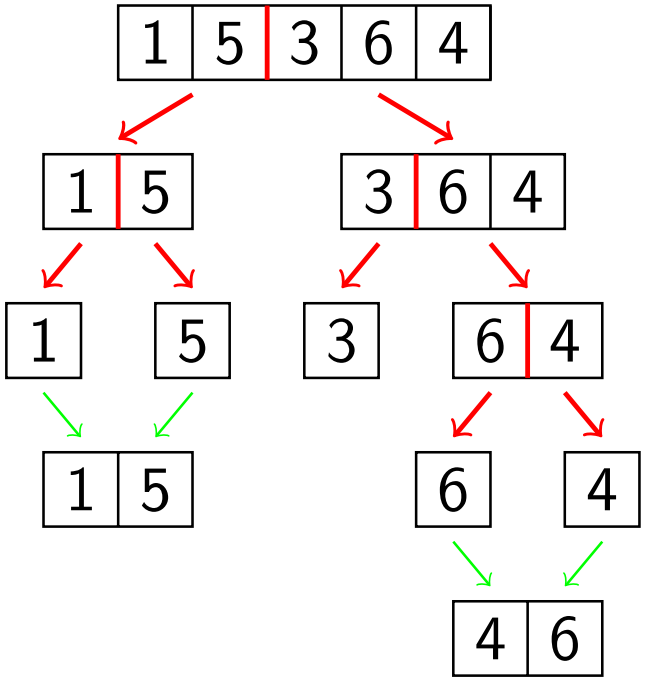
Tri fusion



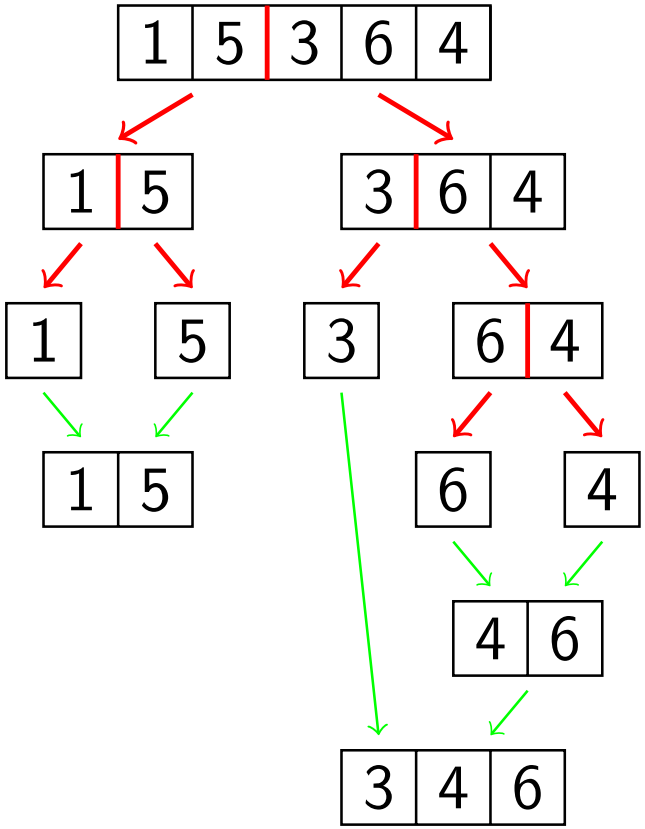
Tri fusion



Tri fusion



Tri fusion



Tri fusion

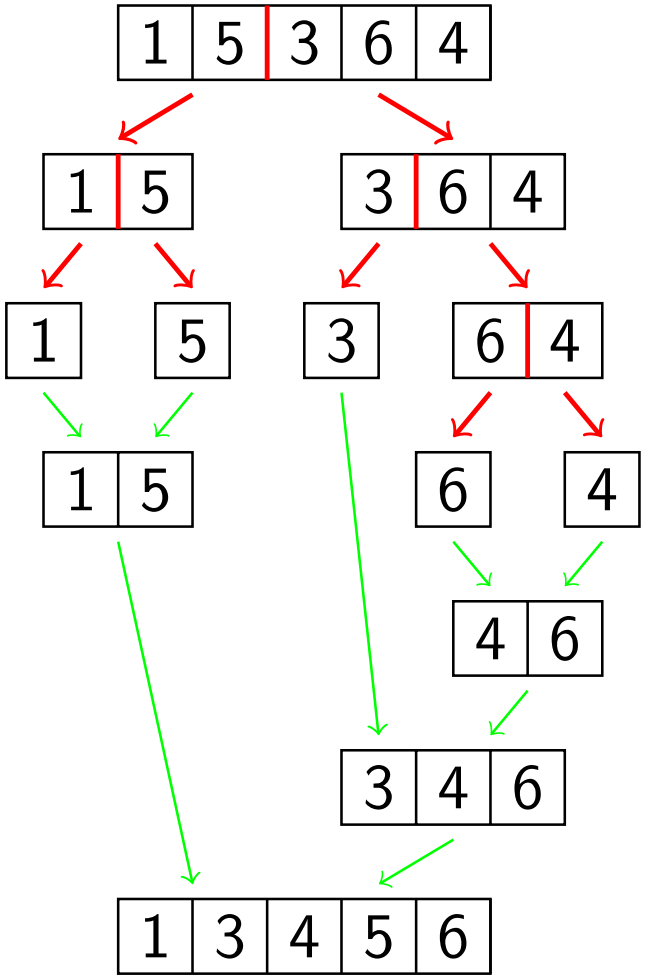


Table des matières

1. Introduction
2. Élément majoritaire
3. Tri fusion
4. Master theorem

Relation avec les récurrences

Un algorithme *diviser pour régner* se traduit souvent par une récurrence :

$$T(n) = a\,T\left(\frac{n}{b}\right) + O(n^d)$$

- $T(n)$: le temps d'exécution au pire des cas sur une entrée de taille n
- a : nombre de sous-problèmes
- n/b : taille de chaque sous-problème
- $O(n^d)$: le polynôme du coût de la division et de la combinaison

Relation avec les récurrences

Un algorithme *diviser pour régner* se traduit souvent par une récurrence :

$$T(n) = a\,T\left(\frac{n}{b}\right) + O(n^d)$$

- $T(n)$: le temps d'exécution au pire des cas sur une entrée de taille n
- a : nombre de sous-problèmes
- n/b : taille de chaque sous-problème
- $O(n^d)$: le polynôme du coût de la division et de la combinaison

Exemple Tri Fusion :

$$T(n) = 2T(n/2) + O(n)$$

Master Théorème

Pour $T(n) = a\,T(n/b) + O(n^d)$:

$$\begin{cases} T(n) = O(n^d) & \text{si } a < b^d \\ T(n) = O(n^d \log_b(n)) \text{ avec } d \geq 0 \text{ une constante} & \text{si } a = b^d \\ T(n) = O(n^{\log_b a}) & \text{si } a > b^d \end{cases}$$

Master Théorème

Pour $T(n) = a\,T(n/b) + O(n^d)$:

$$\begin{cases} T(n) = O(n^d) & \text{si } a < b^d \\ T(n) = O(n^d \log_b(n)) \text{ avec } d \geq 0 \text{ une constante} & \text{si } a = b^d \\ T(n) = O(n^{\log_b a}) & \text{si } a > b^d \end{cases}$$

Interprétation :

- $a < b^d$: la division/combinaison domine
- $a = b^d$: équilibre parfait
- $a > b^d$: la récursion domine

Exemple : Tri Fusion

$$T(n) = 2T(n/2) + O(n)$$

- $a = 2,\ b = 2,\ d = 1$
- Comparer a et b^d : $2 = 2^1$
- On est dans le cas $a = b^d$

Donc :

$$T(n) = O(n \log_b(n)) = O(n \log(n))$$

Démonstration du théorème maître

Démonstration du Master Theorem au Tableau !